

31.1 Basic configuration

The GPDMA is configured using the following registers:

1. Power: In the PCONP register ([Table 46](#)), set bit PCGPDMA.
Remark: On reset, the GPDMA is disabled (PCGPDMA = 0).
2. Clock: see [Table 38](#).
3. Interrupts: Interrupts are enabled in the NVIC using the appropriate Interrupt Set Enable register.
4. Programming: see [Section 31.6](#).

31.2 Introduction

The DMA controller allows peripheral-to memory, memory-to-peripheral, and memory-to-memory transactions. Each DMA stream provides unidirectional serial DMA transfers for a single source and destination. For example, a bi-directional port requires one stream for transmit and one for receives. The source and destination areas can each be either a memory region or a peripheral.

31.3 Features

- Eight DMA channels. Each channel can support an unidirectional transfer.
- 16 DMA request lines.
- Memory-to-memory, memory-to-peripheral, and peripheral-to-memory transfers are supported.
- GPDMA supports the SSP, I2S, UART, A/D Converter, and D/A Converter peripherals. DMA can also be triggered by a timer match condition. Memory-to-memory transfers and transfers to or from GPIO are also supported.
- Scatter or gather DMA is supported through the use of linked lists. This means that the source and destination areas do not have to occupy contiguous areas of memory.
- Hardware DMA channel priority.
- AHB slave DMA programming interface. The DMA Controller is programmed by writing to the DMA control registers over the AHB slave interface.
- One AHB bus master for transferring data. The interface transfers data when a DMA request goes active.
- 32-bit AHB master bus width.
- Incrementing or non-incrementing addressing for source and destination.
- Programmable DMA burst size. The DMA burst size can be programmed to more efficiently transfer data.
- Internal four-word FIFO per channel.
- Supports 8-bit, 16-bit, and 32-bit wide transactions.

- Big-endian and little-endian support. The DMA Controller defaults to little-endian mode on reset.
- An interrupt to the processor can be generated on a DMA completion or when a DMA error has occurred.
- Raw interrupt status. The DMA error and DMA count raw interrupt status can be read prior to masking.
- DMA can operate in Sleep mode. (Note that in Sleep mode the GPDMA cannot access the flash memory).

31.4 Functional description

This section describes the major functional blocks of the DMA Controller.

31.4.1 DMA controller functional description

The DMA Controller enables peripheral-to-memory, memory-to-peripheral, peripheral-to-peripheral, and memory-to-memory transactions. Each DMA stream provides unidirectional serial DMA transfers for a single source and destination. For example, a bidirectional port requires one stream for transmit and one for receive. The source and destination areas can each be either a memory region or a peripheral, and can be accessed through the AHB master. [Figure 135](#) shows a block diagram of the DMA Controller.

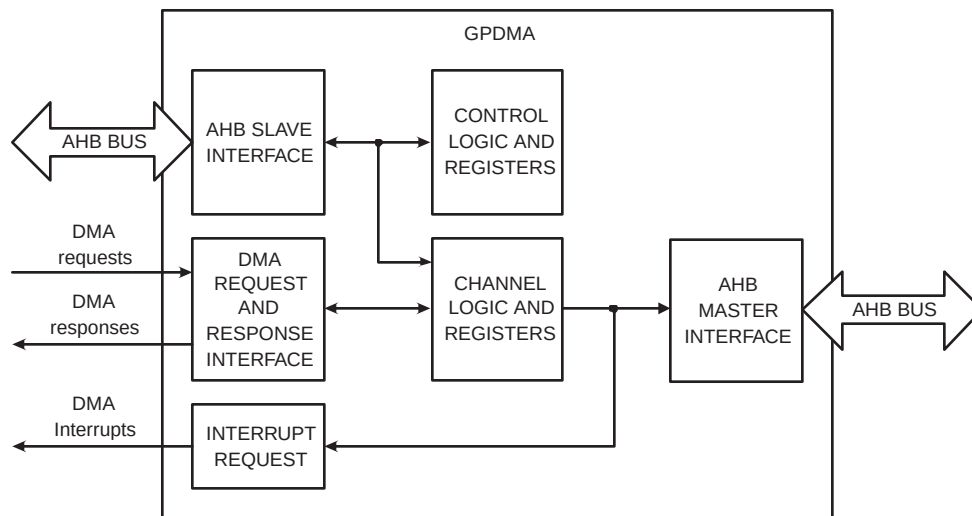


Fig 135. DMA controller block diagram

The functions of the DMA Controller are described in the following sections.

31.4.1.1 AHB slave interface

All transactions to DMA Controller registers on the AHB slave interface are 32 bits wide. 8-bit and 16-bit accesses are not supported and will result in an exception.

31.4.1.2 Control logic and register bank

The register block stores data written or to be read across the AHB interface.

31.4.1.3 DMA request and response interface

See [Section 31.4.2](#) for information on the DMA request and response interface.

31.4.1.4 Channel logic and channel register bank

The channel logic and channel register bank contains registers and logic required for each DMA channel.

31.4.1.5 Interrupt request

The interrupt request generates the interrupt to the ARM processor.

31.4.1.6 AHB master interface

The DMA Controller contains one AHB master interface. The AHB master is capable of dealing with all types of AHB transactions, including:

- Split, retry, and error responses from slaves. If a peripheral performs a split or retry, the DMA Controller stalls and waits until the transaction can complete.
- Locked transfers for source and destination of each stream.
- Setting of protection bits for transfers on each stream.

31.4.1.6.1 Bus and transfer widths

The physical width of the AHB bus is 32 bits. Source and destination transfers can be of differing widths and can be the same width or narrower than the physical bus width. The DMA Controller packs or unpacks data as appropriate.

31.4.1.6.2 Endian behavior

The DMA Controller can cope with both little-endian and big-endian addressing.

Internally the DMA Controller treats all data as a stream of bytes instead of 16-bit or 32-bit quantities. This means that when performing mixed-endian activity, where the endianness of the source and destination are different, byte swapping of the data within the 32-bit data bus is observed.

Note: If byte swapping is not required, then use of different endianness between the source and destination addresses must be avoided. [Table 543](#) shows endian behavior for different source and destination combinations.

Table 543. Endian behavior

Source endian	Destination endian	Source width	Destination width	Source transfer no/byte lane	Source data	Destination transfer no/byte lane	Destination data
Little	Little	8	8	1/[7:0]	21	1/[7:0]	21212121
				2/[15:8]	43	2/[15:8]	43434343
				3/[23:16]	65	3/[23:16]	65656565
				4/[31:24]	87	4/[31:24]	87878787
Little	Little	8	16	1/[7:0]	21	1/[15:0]	43214321
				2/[15:8]	43	2/[31:16]	87658765
				3/[23:16]	65		
				4/[31:24]	87		
Little	Little	8	32	1/[7:0]	21	1/[31:0]	87654321
				2/[15:8]	43		
				3/[23:16]	65		
				4/[31:24]	87		
Little	Little	16	8	1/[7:0]	21	1/[7:0]	21212121
				1/[15:8]	43	2/[15:8]	43434343
				2/[23:16]	65	3/[23:16]	65656565
				2/[31:24]	87	4/[31:24]	87878787
Little	Little	16	16	1/[7:0]	21	1/[15:0]	43214321
				1/[15:8]	43	2/[31:16]	87658765
				2/[23:16]	65		
				2/[31:24]	87		
Little	Little	16	32	1/[7:0]	21	1/[31:0]	87654321
				1/[15:8]	43		
				2/[23:16]	65		
				2/[31:24]	87		
Little	Little	32	8	1/[7:0]	21	1/[7:0]	21212121
				1/[15:8]	43	2/[15:8]	43434343
				1/[23:16]	65	3/[23:16]	65656565
				1/[31:24]	87	4/[31:24]	87878787
Little	Little	32	16	1/[7:0]	21	1/[15:0]	43214321
				1/[15:8]	43	2/[31:16]	87658765
				1/[23:16]	65		
				1/[31:24]	87		
Little	Little	32	32	1/[7:0]	21	1/[31:0]	87654321
				1/[15:8]	43		
				1/[23:16]	65		
				1/[31:24]	87		
Big	Big	8	8	1/[31:24]	12	1/[31:24]	12121212
				2/[23:16]	34	2/[23:16]	34343434
				3/[15:8]	56	3/[15:8]	56565656
				4/[7:0]	78	4/[7:0]	78787878

Table 543. Endian behavior ...continued

Source endian	Destination endian	Source width	Destination width	Source transfer no/byte lane	Source data	Destination transfer no/byte lane	Destination data
Big	Big	8	16	1/[31:24]	12	1/[15:0]	12341234
				2/[23:16]	34	2/[31:16]	56785678
				3/[15:8]	56		
				4/[7:0]	78		
Big	Big	8	32	1/[31:24]	12	1/[31:0]	12345678
				2/[23:16]	34		
				3/[15:8]	56		
				4/[7:0]	78		
Big	Big	16	8	1/[31:24]	12	1/[31:24]	12121212
				1/[23:16]	34	2/[23:16]	34343434
				2/[15:8]	56	3/[15:8]	56565656
				2/[7:0]	78	4/[7:0]	78787878
Big	Big	16	16	1/[31:24]	12	1/[15:0]	12341234
				1/[23:16]	34	2/[31:16]	56785678
				2/[15:8]	56		
				2/[7:0]	78		
Big	Big	16	32	1/[31:24]	12	1/[31:0]	12345678
				1/[23:16]	34		
				2/[15:8]	56		
				2/[7:0]	78		
Big	Big	32	8	1/[31:24]	12	1/[31:24]	12121212
				1/[23:16]	34	2/[23:16]	34343434
				1/[15:8]	56	3/[15:8]	56565656
				1/[7:0]	78	4/[7:0]	78787878
Big	Big	32	16	1/[31:24]	12	1/[15:0]	12341234
				1/[23:16]	34	2/[31:16]	56785678
				1/[15:8]	56		
				1/[7:0]	78		
Big	Big	32	32	1/[31:24]	12	1/[31:0]	12345678
				1/[23:16]	34		
				1/[15:8]	56		
				1/[7:0]	78		

31.4.1.6.3 Error conditions

An error during a DMA transfer is flagged directly by the peripheral by asserting an Error response on the AHB bus during the transfer. The DMA Controller automatically disables the DMA stream after the current transfer has completed, and can optionally generate an error interrupt to the CPU. This error interrupt can be masked.

31.4.1.7 Channel hardware

Each stream is supported by a dedicated hardware channel, including source and destination controllers, as well as a FIFO. This enables better latency than a DMA controller with only a single hardware channel shared between several DMA streams and simplifies the control logic.

31.4.1.8 DMA request priority

DMA channel priority is fixed. DMA channel 0 has the highest priority and DMA channel 7 has the lowest priority.

If the DMA Controller is transferring data for the lower priority channel and then the higher priority channel goes active, it completes the number of transfers delegated to the master interface by the lower priority channel before switching over to transfer data for the higher priority channel. Transfers delegated to the master interface are staged in the DMA channel FIFO, so the amount of data that needs to transfer could be as large as a 4 words.

It is recommended that memory-to-memory transactions use the lowest priority channel.

31.4.1.9 Interrupt generation

A combined interrupt output is generated as an OR function of the individual interrupt requests of the DMA Controller and is connected to the interrupt controller.

31.4.2 DMA system connections

31.4.2.1 DMA request signals

The DMA request signals are used by peripherals to request a data transfer. The DMA request signals indicate whether a single or burst transfer of data is required. The DMA available request signals are:

DMACBREQ[15:0] — Burst request signals. These cause a programmed burst number of data to be transferred.

DMACSREQ[15:0] — Single transfer request signals. These cause a single data to be transferred. The DMA controller transfers a single transfer to or from the peripheral.

DMACLBREQ[15:0] — Last burst request signals.

DMACLSREQ[15:0] — Last single transfer request signals.

Note that peripherals on this device do not support “last” request types, and many do not support both single and burst request types. See [Section 31.4.2.3](#).

31.4.2.2 DMA response signals

The DMA response signals indicate whether the transfer initiated by the DMA request signal has completed. The response signals can also be used to indicate whether a complete packet has been transferred. The DMA response signals from the DMA controller are:

DMACCLR[15:0] — DMA clear or acknowledge signals. The DMACCLR signal is used by the DMA controller to acknowledge a DMA request from the peripheral.

DMACTC[15:0] — DMA terminal count signals. The DMACTC signal can be used by the DMA controller to indicate to the peripheral that the DMA transfer is complete.

31.4.2.3 DMA request connections

The connection of the GPDMA to the supported peripheral devices depends on the DMA functions implemented in those peripherals. [Table 544](#) shows the DMA Request numbers used by the supported peripherals. UART and timer DMA requests on channels 8 through 15 are chosen via the DMAREQSEL register, see [Section 31.5.15](#).

Table 544. DMA Connections

Peripheral Function	DMA Single Request Input (DMACSREQ)	DMA Burst Request Input (DMACBREQ)	DMA Request Signal
SSP0 Tx	0	0	Dedicated DMA requests
SSP0 Rx	1	1	Dedicated DMA requests
SSP1 Tx	2	2	Dedicated DMA requests
SSP1 Rx	3	3	Dedicated DMA requests
ADC	4	4	ADC interrupt request [1]
I ² S channel 0	-	5	Dedicated DMA request
I ² S channel 1	-	6	Dedicated DMA request
DAC	-	7	Dedicated DMA request
UART0 Tx / MAT0.0	-	8	Dedicated DMA requests
UART0 Rx / MAT0.1	-	9	Dedicated DMA requests
UART1 Tx / MAT1.0	-	10	Dedicated DMA requests
UART1 Rx / MAT1.1	-	11	Dedicated DMA requests
UART2 Tx / MAT2.0	-	12	Dedicated DMA requests
UART2 Rx / MAT2.1	-	13	Dedicated DMA requests
UART3 Tx / MAT3.0	-	14	Dedicated DMA requests
UART3 Rx / MAT3.1	-	15	Dedicated DMA requests

[1] Generates an interrupt and/or DMA request depending on software setup.

31.5 Register description

The DMA Controller supports 8 channels. Each channel has registers specific to the operation of that channel. Other registers controls aspects of how source peripherals relate to the DMA Controller. There are also global DMA control and status registers.

The DMA Controller registers are shown in [Table 545](#).

Table 545. GPDMA register map

Name	Description	Access	Reset state	Address
General registers				
DMACIntStat	DMA Interrupt Status Register	RO	0	0x5000 4000
DMACIntTCStat	DMA Interrupt Terminal Count Request Status Register	RO	0	0x5000 4004
DMACIntTCClear	DMA Interrupt Terminal Count Request Clear Register	WO	-	0x5000 4008
DMACIntErrStat	DMA Interrupt Error Status Register	RO	0	0x5000 400C
DMACIntErrClr	DMA Interrupt Error Clear Register	WO	-	0x5000 4010
DMACRawIntTCStat	DMA Raw Interrupt Terminal Count Status Register	RO	0	0x5000 4014
DMACRawIntErrStat	DMA Raw Error Interrupt Status Register	RO	0	0x5000 4018
DMACEnbldChns	DMA Enabled Channel Register	RO	0	0x5000 401C
DMACSoftBReq	DMA Software Burst Request Register	R/W	0	0x5000 4020
DMACSoftSReq	DMA Software Single Request Register	R/W	0	0x5000 4024
DMACSoftLBReq	DMA Software Last Burst Request Register	R/W	0	0x5000 4028
DMACSoftLSReq	DMA Software Last Single Request Register	R/W	0	0x5000 402C
DMACConfig	DMA Configuration Register	R/W	0	0x5000 4030
DMACSync	DMA Synchronization Register	R/W	0	0x5000 4034
DMAREQSEL	Selects between UART and timer DMA requests on channels 8 through 15	R/W	0	0x400F C1C4
Channel 0 registers				
DMACC0SrcAddr	DMA Channel 0 Source Address Register	R/W	0	0x5000 4100
DMACC0DestAddr	DMA Channel 0 Destination Address Register	R/W	0	0x5000 4104
DMACC0LLI	DMA Channel 0 Linked List Item Register	R/W	0	0x5000 4108
DMACC0Control	DMA Channel 0 Control Register	R/W	0	0x5000 410C
DMACC0Config	DMA Channel 0 Configuration Register	R/W	0 [1]	0x5000 4110
Channel 1 registers				
DMACC1SrcAddr	DMA Channel 1 Source Address Register	R/W	0	0x5000 4120
DMACC1DestAddr	DMA Channel 1 Destination Address Register	R/W	0	0x5000 4124
DMACC1LLI	DMA Channel 1 Linked List Item Register	R/W	0	0x5000 4128
DMACC1Control	DMA Channel 1 Control Register	R/W	0	0x5000 412C
DMACC1Config	DMA Channel 1 Configuration Register	R/W	0 [1]	0x5000 4130
Channel 2 registers				
DMACC2SrcAddr	DMA Channel 2 Source Address Register	R/W	0	0x5000 4140
DMACC2DestAddr	DMA Channel 2 Destination Address Register	R/W	0	0x5000 4144
DMACC2LLI	DMA Channel 2 Linked List Item Register	R/W	0	0x5000 4148
DMACC2Control	DMA Channel 2 Control Register	R/W	0	0x5000 414C

Table 545. GPDMA register map

Name	Description	Access	Reset state	Address
DMACC2Config	DMA Channel 2 Configuration Register	R/W	0 [1]	0x5000 4150
Channel 3 registers				
DMACC3SrcAddr	DMA Channel 3 Source Address Register	R/W	0	0x5000 4160
DMACC3DestAddr	DMA Channel 3 Destination Address Register	R/W	0	0x5000 4164
DMACC3LLI	DMA Channel 3 Linked List Item Register	R/W	0	0x5000 4168
DMACC3Control	DMA Channel 3 Control Register	R/W	0	0x5000 416C
DMACC3Config	DMA Channel 3 Configuration Register	R/W	0 [1]	0x5000 4170
Channel 4 registers				
DMACC4SrcAddr	DMA Channel 4 Source Address Register	R/W	0	0x5000 4180
DMACC4DestAddr	DMA Channel 4 Destination Address Register	R/W	0	0x5000 4184
DMACC4LLI	DMA Channel 4 Linked List Item Register	R/W	0	0x5000 4188
DMACC4Control	DMA Channel 4 Control Register	R/W	0	0x5000 418C
DMACC4Config	DMA Channel 4 Configuration Register	R/W	0 [1]	0x5000 4190
Channel 5 registers				
DMACC5SrcAddr	DMA Channel 5 Source Address Register	R/W	0	0x5000 41A0
DMACC5DestAddr	DMA Channel 5 Destination Address Register	R/W	0	0x5000 41A4
DMACC5LLI	DMA Channel 5 Linked List Item Register	R/W	0	0x5000 41A8
DMACC5Control	DMA Channel 5 Control Register	R/W	0	0x5000 41AC
DMACC5Config	DMA Channel 5 Configuration Register	R/W	0 [1]	0x5000 41B0
Channel 6 registers				
DMACC6SrcAddr	DMA Channel 6 Source Address Register	R/W	0	0x5000 41C0
DMACC6DestAddr	DMA Channel 6 Destination Address Register	R/W	0	0x5000 41C4
DMACC6LLI	DMA Channel 6 Linked List Item Register	R/W	0	0x5000 41C8
DMACC6Control	DMA Channel 6 Control Register	R/W	0	0x5000 41CC
DMACC6Config	DMA Channel 6 Configuration Register	R/W	0 [1]	0x5000 41D0
Channel 7 registers				
DMACC7SrcAddr	DMA Channel 7 Source Address Register	R/W	0	0x5000 41E0
DMACC7DestAddr	DMA Channel 7 Destination Address Register	R/W	0	0x5000 41E4
DMACC7LLI	DMA Channel 7 Linked List Item Register	R/W	0	0x5000 41E8
DMACC7Control	DMA Channel 7 Control Register	R/W	0	0x5000 41EC
DMACC7Config	DMA Channel 7 Configuration Register	R/W	0 [1]	0x5000 41F0

[1] Bit 17 of this register is a read-only status flag.

31.5.1 DMA Interrupt Status register (DMACIntStat - 0x5000 4000)

The DMACIntStat Register is read-only and shows the status of the interrupts after masking. A 1 bit indicates that a specific DMA channel interrupt request is active. The request can be generated from either the error or terminal count interrupt requests.

[Table 546](#) shows the bit assignments of the DMACIntStat Register.

Table 546. DMA Interrupt Status register (DMACIntStat - 0x5000 4000)

Bit	Name	Function
7:0	IntStat	Status of DMA channel interrupts after masking. Each bit represents one channel: 0 - the corresponding channel has no active interrupt request. 1 - the corresponding channel does have an active interrupt request.
31:8	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.

31.5.2 DMA Interrupt Terminal Count Request Status register (DMACIntTCStat - 0x5000 4004)

The DMACIntTCStat Register is read-only and indicates the status of the terminal count after masking. [Table 547](#) shows the bit assignments of the DMACIntTCStat Register.

Table 547. DMA Interrupt Terminal Count Request Status register (DMACIntTCStat - 0x5000 4004)

Bit	Name	Function
7:0	IntTCStat	Terminal count interrupt request status for DMA channels. Each bit represents one channel: 0 - the corresponding channel has no active terminal count interrupt request. 1 - the corresponding channel does have an active terminal count interrupt request.
31:8	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.

31.5.3 DMA Interrupt Terminal Count Request Clear register (DMACIntTCClear - 0x5000 4008)

The DMACIntTCClear Register is write-only and clears one or more terminal count interrupt requests. When writing to this register, each data bit that contains a 1 causes the corresponding bit in the status register (DMACIntTCStat) to be cleared. Data bits that are 0 have no effect. [Table 548](#) shows the bit assignments of the DMACIntTCClear Register.

Table 548. DMA Interrupt Terminal Count Request Clear register (DMACIntTCClear - 0x5000 4008)

Bit	Name	Function
7:0	IntTCClear	Allows clearing the Terminal count interrupt request (IntTCStat) for DMA channels. Each bit represents one channel: 0 - writing 0 has no effect. 1 - clears the corresponding channel terminal count interrupt.
31:8	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.

31.5.4 DMA Interrupt Error Status register (DMACIntErrStat - 0x5000 400C)

The DMACIntErrStat Register is read-only and indicates the status of the error request after masking. [Table 549](#) shows the bit assignments of the DMACIntErrStat Register.

Table 549. DMA Interrupt Error Status register (DMACIntErrStat - 0x5000 400C)

Bit	Name	Function
7:0	IntErrStat	Interrupt error status for DMA channels. Each bit represents one channel: 0 - the corresponding channel has no active error interrupt request. 1 - the corresponding channel does have an active error interrupt request.
31:8	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.

31.5.5 DMA Interrupt Error Clear register (DMACIntErrClr - 0x5000 4010)

The DMACIntErrClr Register is write-only and clears the error interrupt requests. When writing to this register, each data bit that is 1 causes the corresponding bit in the status register to be cleared. Data bits that are 0 have no effect on the corresponding bit in the register. [Table 550](#) shows the bit assignments of the DMACIntErrClr Register.

Table 550. DMA Interrupt Error Clear register (DMACIntErrClr - 0x5000 4010)

Bit	Name	Function
7:0	IntErrClr	Writing a 1 clears the error interrupt request (IntErrStat) for DMA channels. Each bit represents one channel: 0 - writing 0 has no effect. 1 - clears the corresponding channel error interrupt.
31:8	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.

31.5.6 DMA Raw Interrupt Terminal Count Status register (DMACRawIntTCStat - 0x5000 4014)

The DMACRawIntTCStat Register is read-only and indicates which DMA channel is requesting a transfer complete (terminal count interrupt) prior to masking. (Note: the DMACIntTCStat Register contains the same information after masking.) A 1 bit indicates that the terminal count interrupt request is active prior to masking. [Table 551](#) shows the bit assignments of the DMACRawIntTCStat Register.

Table 551. DMA Raw Interrupt Terminal Count Status register (DMACRawIntTCStat - 0x5000 4014)

Bit	Name	Function
7:0	RawIntTCStat	Status of the terminal count interrupt for DMA channels prior to masking. Each bit represents one channel: 0 - the corresponding channel has no active terminal count interrupt request. 1 - the corresponding channel does have an active terminal count interrupt request.
31:8	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.

31.5.7 DMA Raw Error Interrupt Status register (DMACRawIntErrStat - 0x5000 4018)

The DMACRawIntErrStat Register is read-only and indicates which DMA channel is requesting an error interrupt prior to masking. (Note: the DMACIntErrStat Register contains the same information after masking.) A 1 bit indicates that the error interrupt request is active prior to masking. [Table 552](#) shows the bit assignments of register of the DMACRawIntErrStat Register.

Table 552. DMA Raw Error Interrupt Status register (DMACRawIntErrStat - 0x5000 4018)

Bit	Name	Function
7:0	RawIntErrStat	Status of the error interrupt for DMA channels prior to masking. Each bit represents one channel: 0 - the corresponding channel has no active error interrupt request. 1 - the corresponding channel does have an active error interrupt request.
31:8	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.

31.5.8 DMA Enabled Channel register (DMACEnbldChns - 0x5000 401C)

The DMACEnbldChns Register is read-only and indicates which DMA channels are enabled, as indicated by the Enable bit in the DMACCxConfig Register. A 1 bit indicates that a DMA channel is enabled. A bit is cleared on completion of the DMA transfer.

[Table 553](#) shows the bit assignments of the DMACEnbldChns Register.

Table 553. DMA Enabled Channel register (DMACEnbldChns - 0x5000 401C)

Bit	Name	Function
7:0	EnabledChannels	Enable status for DMA channels. Each bit represents one channel: 0 - DMA channel is disabled. 1 - DMA channel is enabled.
31:8	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.

31.5.9 DMA Software Burst Request register (DMACSoftBReq - 0x5000 4020)

The DMACSoftBReq Register is read/write and enables DMA burst requests to be generated by software. A DMA request can be generated for each source by writing a 1 to the corresponding register bit. A register bit is cleared when the transaction has completed. Reading the register indicates which sources are requesting DMA burst transfers. A request can be generated from either a peripheral or the software request register. Each bit is cleared when the related transaction has completed. [Table 554](#) shows the bit assignments of the DMACSoftBReq Register.

Table 554. DMA Software Burst Request register (DMACSoftBReq - 0x5000 4020)

Bit	Name	Function
15:0	SoftBReq	Software burst request flags for each of 16 possible sources. Each bit represents one DMA request line or peripheral function (refer to Table 544 for peripheral hardware connections to the DMA controller): 0 - writing 0 has no effect. 1 - writing 1 generates a DMA burst request for the corresponding request line.
31:16	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.

Note: It is recommended that software and hardware peripheral requests are not used at the same time.

31.5.10 DMA Software Single Request register (DMACSoftSReq - 0x5000 4024)

The DMACSoftSReq Register is read/write and enables DMA single transfer requests to be generated by software. A DMA request can be generated for each source by writing a 1 to the corresponding register bit. A register bit is cleared when the transaction has completed. Reading the register indicates which sources are requesting single DMA transfers. A request can be generated from either a peripheral or the software request register. [Table 555](#) shows the bit assignments of the DMACSoftSReq Register.

Table 555. DMA Software Single Request register (DMACSoftSReq - 0x5000 4024)

Bit	Name	Function
15:0	SoftSReq	Software single transfer request flags for each of 16 possible sources. Each bit represents one DMA request line or peripheral function: 0 - writing 0 has no effect. 1 - writing 1 generates a DMA single transfer request for the corresponding request line.
31:16	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.

31.5.11 DMA Software Last Burst Request register (DMACSoftLBReq - 0x5000 4028)

The DMACSoftLBReq Register is read/write and enables DMA last burst requests to be generated by software. A DMA request can be generated for each source by writing a 1 to the corresponding register bit. A register bit is cleared when the transaction has completed. Reading the register indicates which sources are requesting last burst DMA transfers. A request can be generated from either a peripheral or the software request register. [Table 556](#) shows the bit assignments of the DMACSoftLBReq Register.

Table 556. DMA Software Last Burst Request register (DMACSoftLBReq - 0x5000 4028)

Bit	Name	Function
15:0	SoftLBReq	Software last burst request flags for each of 16 possible sources. Each bit represents one DMA request line or peripheral function: 0 - writing 0 has no effect. 1 - writing 1 generates a DMA last burst request for the corresponding request line.
31:16	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.

31.5.12 DMA Software Last Single Request register (DMACSoftLSReq - 0x5000 402C)

The DMACSoftLSReq Register is read/write and enables DMA last single requests to be generated by software. A DMA request can be generated for each source by writing a 1 to the corresponding register bit. A register bit is cleared when the transaction has completed. Reading the register indicates which sources are requesting last single DMA transfers. A request can be generated from either a peripheral or the software request register. [Table 557](#) shows the bit assignments of the DMACSoftLSReq Register.

Table 557. DMA Software Last Single Request register (DMACSoftLSReq - 0x5000 402C)

Bit	Name	Function
15:0	SoftLSReq	Software last single transfer request flags for each of 16 possible sources. Each bit represents one DMA request line or peripheral function: 0 - writing 0 has no effect. 1 - writing 1 generates a DMA last single transfer request for the corresponding request line.
31:16	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.

31.5.13 DMA Configuration register (DMACConfig - 0x5000 4030)

The DMACConfig Register is read/write and configures the operation of the DMA Controller. The endianness of the AHB master interface can be altered by writing to the M bit of this register. The AHB master interface is set to little-endian mode on reset.

[Table 558](#) shows the bit assignments of the DMACConfig Register.

Table 558. DMA Configuration register (DMACConfig - 0x5000 4030)

Bit	Name	Function
0	E	DMA Controller enable: 0 = disabled (default). Disabling the DMA Controller reduces power consumption. 1 = enabled.
1	M	AHB Master endianness configuration: 0 = little-endian mode (default). 1 = big-endian mode.
31:2	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.

31.5.14 DMA Synchronization register (DMACSync - 0x5000 4034)

The DMACSync Register is read/write and enables or disables synchronization logic for the DMA request signals. The DMA request signals consist of the DMACBREQ[15:0], DMACSREQ[15:0], DMACLBREQ[15:0], and DMACLSREQ[15:0]. A bit set to 0 enables the synchronization logic for a particular group of DMA requests. A bit set to 1 disables the synchronization logic for a particular group of DMA requests. This register is reset to 0, enabling synchronization logic by default. [Table 559](#) shows the bit assignments of the DMACSync Register.

Table 559. DMA Synchronization register (DMACSync - 0x5000 4034)

Bit	Name	Function
15:0	DMACSync	Controls the synchronization logic for DMA request signals. Each bit represents one set of DMA request lines as described in the preceding text: 0 - synchronization logic for the corresponding DMA request signals are enabled. 1 - synchronization logic for the corresponding request line signals are disabled.
31:16	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.

31.5.15 DMA Request Select register (DMAReqSel - 0x400F C1C4)

DMAReqSel is a read/write register that allows selecting between UART or Timer DMA requests for DMA inputs 8 through 15. [Table 560](#) shows the bit assignments of the DMAReqSel Register.

Table 560. DMA Request Select register (DMAReqSel - 0x400F C1C4)

Bit	Name	Function
0	DMASEL08	Selects the DMA request for GPDMA input 8: 0 - UART0 TX is selected. 1 - Timer 0 match 0 is selected.
1	DMASEL09	Selects the DMA request for GPDMA input 9: 0 - UART0 RX is selected. 1 - Timer 0 match 1 is selected.
2	DMASEL10	Selects the DMA request for GPDMA input 10: 0 - UART1 TX is selected. 1 - Timer 1match 0 is selected.
3	DMASEL11	Selects the DMA request for GPDMA input 11: 0 - UART1 RX is selected. 1 - Timer 1match 1 is selected.
4	DMASEL12	Selects the DMA request for GPDMA input 12: 0 - UART2 TX is selected. 1 - Timer 2 match 0 is selected.
5	DMASEL13	Selects the DMA request for GPDMA input 13: 0 - UART2 RX is selected. 1 - Timer 2 match 1 is selected.
6	DMASEL14	Selects the DMA request for GPDMA input 14: 0 - UART3 TX is selected. 1 - Timer 3 match 0 is selected.
7	DMASEL15	Selects the DMA request for GPDMA input 15: 0 - UART3 RX is selected. 1 - Timer 3 match 1 is selected.
31:8	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.

31.5.16 DMA Channel registers

The channel registers are used to program the eight DMA channels. These registers consist of:

- Eight DMACCxSrcAddr Registers.
- Eight DMACCxDestAddr Registers.
- Eight DMACCxLLI Registers.
- Eight DMACCxControl Registers.
- Eight DMACCxConfig Registers.

When performing scatter/gather DMA, the first four of these are automatically updated.

31.5.17 DMA Channel Source Address registers (DMACCxSrcAddr - 0x5000 41x0)

The eight read/write DMACCxSrcAddr Registers (DMACC0SrcAddr to DMACC7SrcAddr) contain the current source address (byte-aligned) of the data to be transferred. Each register is programmed directly by software before the appropriate channel is enabled. When the DMA channel is enabled this register is updated:

- As the source address is incremented.
- By following the linked list when a complete packet of data has been transferred.

Reading the register when the channel is active does not provide useful information. This is because by the time software has processed the value read, the address may have progressed. It is intended to be read-only when the channel has stopped, in which case it shows the source address of the last item read.

Note: The source and destination addresses must be aligned to the source and destination widths.

[Table 561](#) shows the bit assignments of the DMACCxSrcAddr Registers.

Table 561. DMA Channel Source Address registers (DMACCxSrcAddr - 0x5000 41x0)

Bit	Name	Function
31:0	SrcAddr	DMA source address. Reading this register will return the current source address.

31.5.18 DMA Channel Destination Address registers (DMACCxDestAddr - 0x5000 41x4)

The eight read/write DMACCxDestAddr Registers (DMACC0DestAddr to DMACC7DestAddr) contain the current destination address (byte-aligned) of the data to be transferred. Each register is programmed directly by software before the channel is enabled. When the DMA channel is enabled the register is updated as the destination address is incremented and by following the linked list when a complete packet of data has been transferred. Reading the register when the channel is active does not provide useful information. This is because by the time that software has processed the value read, the address may have progressed. It is intended to be read-only when a channel has stopped, in which case it shows the destination address of the last item read.

[Table 562](#) shows the bit assignments of the DMACCxDestAddr Register.

Table 562. DMA Channel Destination Address registers (DMACCxDestAddr - 0x5000 41x4)

Bit	Name	Function
31:0	DestAddr	DMA Destination address. Reading this register will return the current destination address.

31.5.19 DMA Channel Linked List Item registers (DMACCxLLI - 0x5000 41x8)

The eight read/write DMACCxLLI Registers (DMACC0LLI to DMACC7LLI) contain a word-aligned address of the next Linked List Item (LLI). If the LLI is 0, then the current LLI is the last in the chain, and the DMA channel is disabled when all DMA transfers associated with it are completed. Programming this register when the DMA channel is enabled may have unpredictable side effects. [Table 563](#) shows the bit assignments of the DMACCxLLI Register.

Table 563. DMA Channel Linked List Item registers (DMACCxLLI - 0x5000 41x8)

Bit	Name	Function
1:0	-	Reserved, and must be written as 0.
31:2	LLI	Linked list item. Bits [31:2] of the address for the next LLI. Address bits [1:0] are 0.

31.5.20 DMA channel control registers (DMACCxControl - 0x5000 41xC)

The eight read/write DMACCxControl Registers (DMACC0Control to DMACC7Control) contain DMA channel control information such as the transfer size, burst size, and transfer width. Each register is programmed directly by software before the DMA channel is enabled. When the channel is enabled the register is updated by following the linked list when a complete packet of data has been transferred. Reading the register while the channel is active does not give useful information. This is because by the time software has processed the value read, the channel may have advanced. It is intended to be read-only when a channel has stopped. [Table 564](#) shows the bit assignments of the DMACCxControl Register.

31.5.20.1 Protection and access information

AHB access information is provided to the source and/or destination peripherals when a transfer occurs, although on the LPC176x/5x this has no effect. The transfer information is provided by programming the DMA channel (the Prot bits of the DMACCxControl Register, and the Lock bit of the DMACCxConfig Register). These bits are programmed by software, and can be used by peripherals. Three bits of information are provided, and are used as shown in [Table 564](#).

Table 564. DMA channel control registers (DMACCxControl - 0x5000 41xC)

Bit	Name	Function
11:0	TransferSize	Transfer size. This field sets the size of the transfer. The transfer size value must be set before the channel is enabled. Transfer size is updated as data transfers are completed. A read from this field indicates the number of transfers completed on the destination bus. Reading the register when the channel is active does not give useful information because by the time that the software has processed the value read, the channel might have progressed. It is intended to be used only when a channel is enabled and then disabled.
14:12	SBSize	Source burst size. Indicates the number of transfers that make up a source burst. This value must be set to the burst size of the source peripheral, or if the source is memory, to the memory boundary size. The burst size is the amount of data that is transferred when the DMACBREQ signal goes active in the source peripheral. 000 - 1 001 - 4 010 - 8 011 - 16 100 - 32 101 - 64 110 - 128 111 - 256
17:15	DBSize	Destination burst size. Indicates the number of transfers that make up a destination burst transfer request. This value must be set to the burst size of the destination peripheral or, if the destination is memory, to the memory boundary size. The burst size is the amount of data that is transferred when the DMACBREQ signal goes active in the destination peripheral. 000 - 1 001 - 4 010 - 8 011 - 16 100 - 32 101 - 64 110 - 128 111 - 256
20:18	SWidth	Source transfer width. Transfers wider than the AHB master bus width are illegal. The source and destination widths can be different from each other. The hardware automatically packs and unpacks the data as required. 000 - Byte (8-bit) 001 - Halfword (16-bit) 010 - Word (32-bit) 011 to 111 - Reserved
23:21	DWidth	Destination transfer width. Transfers wider than the AHB master bus width are not supported. The source and destination widths can be different from each other. The hardware automatically packs and unpacks the data as required. 000 - Byte (8-bit) 001 - Halfword (16-bit) 010 - Word (32-bit) 011 to 111 - Reserved
25:24	-	Reserved, and must be written as 0.

Table 564. DMA channel control registers (DMACCxControl - 0x5000 41xC) ...continued

Bit	Name	Function
26	SI	Source increment: 0 - the source address is not incremented after each transfer. 1 - the source address is incremented after each transfer.
27	DI	Destination increment: 0 - the destination address is not incremented after each transfer. 1 - the destination address is incremented after each transfer.
28	Prot1	This is provided to the peripheral during a DMA bus access and indicates that the access is in user mode or privileged mode. This information is not used in the LPC176x/5x. 0 - access is in user mode. 1 - access is in privileged mode.
29	Prot2	This is provided to the peripheral during a DMA bus access and indicates to the peripheral that the access is bufferable or not bufferable. This information is not used in the LPC176x/5x. 0 - access is not bufferable. 1 - access is bufferable.
30	Prot3	This is provided to the peripheral during a DMA bus access and indicates to the peripheral that the access is cacheable or not cacheable. This information is not used in the LPC176x/5x. 0 - access is not cacheable. 1 - access is cacheable.
31	I	Terminal count interrupt enable bit. 0 - the terminal count interrupt is disabled. 1 - the terminal count interrupt is enabled.

31.5.21 DMA Channel Configuration registers (DMACCxConfig - 0x5000 41x0)

The eight DMACCxConfig Registers (DMACC0Config to DMACC7Config) are read/write with the exception of bit[17] which is read-only. These registers configure each DMA channel. The registers are not updated when a new LLI is requested. [Table 565](#) shows the bit assignments of the DMACCxConfig Register.

Table 565. DMA Channel Configuration registers (DMACCxConfig - 0x5000 41x0)

Bit	Name	Function
0	E	Channel enable. Reading this bit indicates whether a channel is currently enabled or disabled: 0 = channel disabled. 1 = channel enabled. The Channel Enable bit status can also be found by reading the DMACEnbldChns Register. A channel is enabled by setting this bit. A channel can be disabled by clearing the Enable bit. This causes the current AHB transfer (if one is in progress) to complete and the channel is then disabled. Any data in the FIFO of the relevant channel is lost. Restarting the channel by setting the Channel Enable bit has unpredictable effects, the channel must be fully re-initialized. The channel is also disabled, and Channel Enable bit cleared, when the last LLI is reached, the DMA transfer is completed, or if a channel error is encountered. If a channel must be disabled without losing data in the FIFO, the Halt bit must be set so that further DMA requests are ignored. The Active bit must then be polled until it reaches 0, indicating that there is no data left in the FIFO. Finally, the Channel Enable bit can be cleared.
5:1	SrcPeripheral	Source peripheral. This value selects the DMA source request peripheral. This field is ignored if the source of the transfer is from memory. See Table 544 for peripheral identification.
10:6	DestPeripheral	Destination peripheral. This value selects the DMA destination request peripheral. This field is ignored if the destination of the transfer is to memory. See Table 544 for peripheral identification.
13:11	TransferType	This value indicates the type of transfer. The transfer type can be memory-to-memory, memory-to-peripheral, peripheral-to-memory, or peripheral-to-peripheral. Refer to Table 566 for the encoding of this field.
14	IE	Interrupt error mask. When cleared, this bit masks out the error interrupt of the relevant channel.
15	ITC	Terminal count interrupt mask. When cleared, this bit masks out the terminal count interrupt of the relevant channel.
16	L	Lock. When set, this bit enables locked transfers. This information is not used in the LPC176x/5x.
17	A	Active: 0 = there is no data in the FIFO of the channel. 1 = the channel FIFO has data. This value can be used with the Halt and Channel Enable bits to cleanly disable a DMA channel. This is a read-only bit.
18	H	Halt: 0 = enable DMA requests. 1 = ignore further source DMA requests. The contents of the channel FIFO are drained. This value can be used with the Active and Channel Enable bits to cleanly disable a DMA channel.
31:19	Reserved	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.

31.5.21.1 Lock control

The lock control may set the lock bit by writing a 1 to bit 16 of the DMACCxConfig Register. When a burst occurs, the AHB arbiter will not de-grant the master during the burst until the lock is de-asserted. The DMA Controller can be locked for a single burst such as a long source fetch burst or a long destination drain burst. The DMA Controller does not usually assert the lock continuously for a source fetch burst followed by a destination drain burst.

There are situations when the DMA Controller asserts the lock for source transfers followed by destination transfers. This is possible when internal conditions in the DMA Controller permit it to perform a source fetch followed by a destination drain back-to-back.

31.5.21.2 Transfer type

[Table 566](#) lists the bit values of the transfer type bits identified in [Table 565](#).

Table 566. Transfer type bits

Bit value	Transfer type	Controller
000	Memory to memory	DMA
001	Memory to peripheral	DMA
010	Peripheral to memory	DMA
011	Source peripheral to destination peripheral	DMA
100 to 111	Reserved, do not use these combinations	-

31.6 Using the DMA controller

31.6.1 Programming the DMA controller

All accesses to the DMA Controller internal register must be word (32-bit) reads and writes.

31.6.1.1 Enabling the DMA controller

To enable the DMA controller set the Enable bit in the DMACConfig register.

31.6.1.2 Disabling the DMA controller

To disable the DMA controller:

- Read the DMACEnbldChns register and ensure that all the DMA channels have been disabled. If any channels are active, see Disabling a DMA channel.
- Disable the DMA controller by writing 0 to the DMA Enable bit in the DMACConfig register.

31.6.1.3 Enabling a DMA channel

To enable the DMA channel set the channel enable bit in the relevant DMA channel configuration register. Note that the channel must be fully initialized before it is enabled.

31.6.1.4 Disabling a DMA channel

A DMA channel can be disabled in three ways:

- By writing directly to the channel enable bit. Any outstanding data in the FIFO's is lost if this method is used.
- By using the active and halt bits in conjunction with the channel enable bit.
- By waiting until the transfer completes. This automatically clears the channel.

Disabling a DMA channel and losing data in the FIFO

Clear the relevant channel enable bit in the relevant channel configuration register. The current AHB transfer (if one is in progress) completes and the channel is disabled. Any data in the FIFO is lost.

Disabling the DMA channel without losing data in the FIFO

- Set the halt bit in the relevant channel configuration register. This causes any future DMA request to be ignored.
- Poll the active bit in the relevant channel configuration register until it reaches 0. This bit indicates whether there is any data in the channel that has to be transferred.
- Clear the channel enable bit in the relevant channel configuration register

31.6.1.5 Setting up a new DMA transfer

To set up a new DMA transfer:

If the channel is not set aside for the DMA transaction:

1. Read the DMACEnbldChns controller register and find out which channels are inactive.
2. Choose an inactive channel that has the required priority.
3. Program the DMA controller

31.6.1.6 Halting a DMA channel

Set the halt bit in the relevant DMA channel configuration register. The current source request is serviced. Any further source DMA request is ignored until the halt bit is cleared.

31.6.1.7 Programming a DMA channel

1. Choose a free DMA channel with the priority needed. DMA channel 0 has the highest priority and DMA channel 7 the lowest priority.
2. Clear any pending interrupts on the channel to be used by writing to the DMACIntTCClear and DMACIntErrClear register. The previous channel operation might have left interrupt active.
3. Write the source address into the DMACCxSrcAddr register.
4. Write the destination address into the DMACCxDestAddr register.
5. Write the address of the next LLI into the DMACCxLLI register. If the transfer comprises of a single packet of data then 0 must be written into this register.
6. Write the control information into the DMACCxControl register.
7. Write the channel configuration information into the DMACCxConfig register. If the enable bit is set then the DMA channel is automatically enabled.

31.6.2 Flow control

The device that controls the length of the packet is known as the flow controller. On the LPC176x/5x, the flow controller is always the DMA Controller, and the packet length is programmed by software before the DMA channel is enabled.

When the DMA transfer is completed:

1. The DMA Controller issues an acknowledge to the peripheral in order to indicate that the transfer has finished.
2. A TC interrupt is generated, if enabled.
3. The DMA Controller moves on to the next LLI.

The following sections describe the DMA Controller data flow sequences for the four allowed transfer types:

- Memory-to-peripheral.
- Peripheral-to-memory.
- Memory-to-memory.
- Peripheral-to-peripheral.

[Table 567](#) indicates the request signals used for each type of transfer.

Table 567. DMA request signal usage

Transfer direction	Request generator	Flow controller
Memory-to-peripheral	Peripheral	DMA Controller
Peripheral-to-memory	Peripheral	DMA Controller
Memory-to-memory	DMA Controller	DMA Controller
Source peripheral to destination peripheral	Source peripheral and destination peripheral	DMA Controller

31.6.2.1 Peripheral-to-memory or memory-to-peripheral DMA flow

For a peripheral-to-memory or memory-to-peripheral DMA flow, the following sequence occurs:

1. Program and enable the DMA channel.
2. Wait for a DMA request.
3. The DMA Controller starts transferring data when:
 - The DMA request goes active.
 - The DMA stream has the highest pending priority.
 - The DMA Controller is the bus master of the AHB bus.
4. If an error occurs while transferring the data, an error interrupt is generated and disables the DMA stream, and the flow sequence ends.
5. Decrement the transfer count.
6. If the transfer has completed (indicated by the transfer count reaching 0):
 - The DMA Controller responds with a DMA acknowledge.
 - The terminal count interrupt is generated (this interrupt can be masked).
 - If the DMACCxLLI Register is not 0, then reload the DMACCxSrcAddr, DMACCxDestAddr, DMACCxLLI, and DMACCxControl registers and go to back to step 2. However, if DMACCxLLI is 0, the DMA stream is disabled and the flow sequence ends.

31.6.2.2 Peripheral-to-peripheral DMA flow

For a peripheral-to-peripheral DMA flow, the following sequence occurs:

1. Program and enable the DMA channel.
2. Wait for a source DMA request.
3. The DMA Controller starts transferring data when:
 - The DMA request goes active.
 - The DMA stream has the highest pending priority.
 - The DMA Controller is the bus master of the AHB bus.
4. If an error occurs while transferring the data an error interrupt is generated, the DMA stream is disabled, and the flow sequence ends.
5. Decrement the transfer count.
6. If the transfer has completed (indicated by the transfer count reaching 0):
 - The DMA Controller responds with a DMA acknowledge to the source peripheral.
 - Further source DMA requests are ignored.

7. When the destination DMA request goes active and there is data in the DMA Controller FIFO, transfer data into the destination peripheral.
8. If an error occurs while transferring the data, an error interrupt is generated, the DMA stream is disabled, and the flow sequence ends.
9. If the transfer has completed it is indicated by the transfer count reaching 0. The following happens:
 - The DMA Controller responds with a DMA acknowledge to the destination peripheral.
 - The terminal count interrupt is generated (this interrupt can be masked).
 - If the DMACCxLLI Register is not 0, then reload the DMACCxSrcAddr, DMACCxDestAddr, DMACCxLLI, and DMACCxControl Registers and go back to step 2. However, if DMACCxLLI is 0, the DMA stream is disabled and the flow sequence ends.

31.6.2.3 Memory-to-memory DMA flow

For a memory-to-memory DMA flow the following sequence occurs:

1. Program and enable the DMA channel.
2. Transfer data whenever the DMA channel has the highest pending priority and the DMA Controller gains mastership of the AHB bus.
3. If an error occurs while transferring the data, generate an error interrupt and disable the DMA stream.
4. Decrement the transfer count.
5. If the count has reached zero:
 - Generate a terminal count interrupt (the interrupt can be masked).
 - If the DMACCxLLI Register is not 0, then reload the DMACCxSrcAddr, DMACCxDestAddr, DMACCxLLI, and DMACCxControl Registers and go back to step 2. However, if DMACCxLLI is 0, the DMA stream is disabled and the flow sequence ends.

Note: Memory-to-memory transfers should be programmed with a low channel priority, otherwise other DMA channels cannot access the bus until the memory-to-memory transfer has finished, or other AHB masters cannot perform any transaction.

31.6.3 Interrupt requests

Interrupt requests can be generated when an AHB error is encountered or at the end of a transfer (terminal count), after all the data corresponding to the current LLI has been transferred to the destination. The interrupts can be masked by programming bits in the relevant DMACCxControl and DMACCxConfig Channel Registers. The interrupt requests from all DMA channels can be found in the DMACRawIntTCStat and DMACRawIntErrStat registers. The masked versions of the DMA interrupt data is contained in the DMACIntTCStat and DMACIntErrStat registers. The DMACIntStat register then combines the DMACIntTCStat and DMACIntErrStat requests into a single register to enable the source of an interrupt to be found quickly. Writing to the DMACIntTCClear or the DMACIntErrClr Registers with a bit set to 1 enables selective clearing of interrupts.

31.6.3.1 Hardware interrupt sequence flow

When a DMA interrupt request occurs, the Interrupt Service Routine needs to:

1. Read the DMACIntTCStat Register to determine whether the interrupt was generated due to the end of the transfer (terminal count). A 1 bit indicates that the transfer completed. If more than one request is active, it is recommended that the highest priority channels be checked first.
2. Read the DMACIntErrStat Register to determine whether the interrupt was generated due to an error occurring. A 1 bit indicates that an error occurred.
3. Service the interrupt request.
4. For a terminal count interrupt, write a 1 to the relevant bit of the DMACIntTCClr Register. For an error interrupt write a 1 to the relevant bit of the DMACIntErrClr Register to clear the interrupt request.

31.6.4 Address generation

Address generation can be either incrementing or non-incrementing (address wrapping is not supported).

Some devices, especially memories, disallow burst accesses across certain address boundaries. The DMA controller assumes that this is the case with any source or destination area, which is configured for incrementing addressing. This boundary is assumed to be aligned with the specified burst size. For example, if the channel is set for 16-transfer burst to a 32-bit wide device then the boundary is 64-bytes aligned (that is address bits [5:0] equal 0). If a DMA burst is to cross one of these boundaries, then, instead of a burst, that transfer is split into separate AHB transactions.

31.6.4.1 Word-aligned transfers across a boundary

The channel is configured for 16-transfer bursts, each transfer 32-bits wide, to a destination for which address incrementing is enabled. The start address for the current burst is 0x0C000024, the next boundary (calculated from the burst size and transfer width) is 0x0C000040.

The transfer will be split into two AHB transactions:

- a 7-transfer burst starting at address 0x0C000024
- a 9-transfer burst starting at address 0x0C000040.

31.6.5 Scatter/gather

Scatter/gather is supported through the use of linked lists. This means that the source and destination areas do not have to occupy contiguous areas in memory. Where scatter/gather is not required, the DMAACCxLLI Register must be set to 0.

The source and destination data areas are defined by a series of linked lists. Each Linked List Item (LLI) controls the transfer of one block of data, and then optionally loads another LLI to continue the DMA operation, or stops the DMA stream. The first LLI is programmed into the DMA Controller.

The data to be transferred described by an LLI (referred to as the packet of data) usually requires one or more DMA bursts (to each of the source and destination).

31.6.5.1 Linked list items

A Linked List Item (LLI) consists of four words. These words are organized in the following order:

1. DMACCxSrcAddr.
2. DMACCxDestAddr.
3. DMACCxLLI.
4. DMACCxControl.

Note: The DMACCxConfig DMA channel Configuration Register is not part of the linked list item.

31.6.5.1.1 Programming the DMA controller for scatter/gather DMA

To program the DMA Controller for scatter/gather DMA:

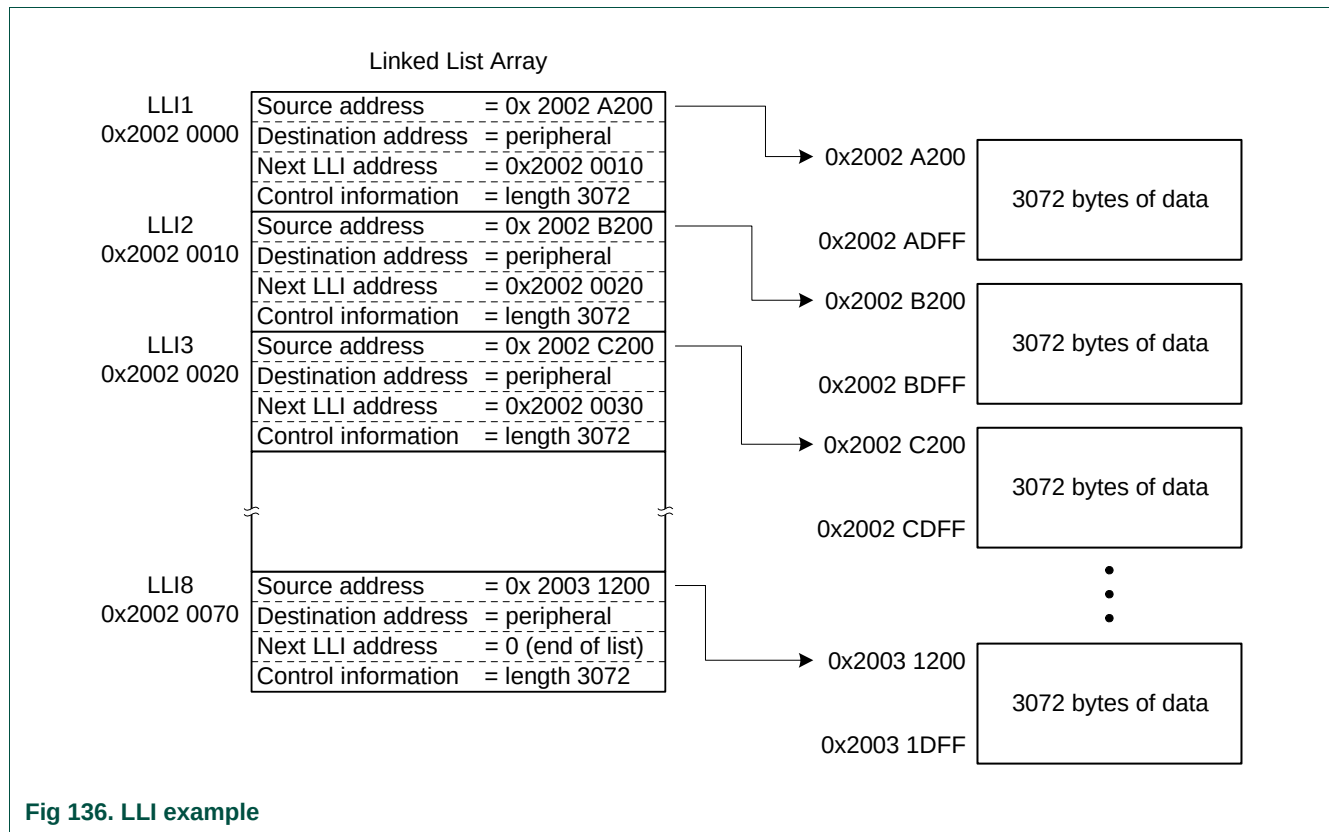
1. Write the LLIs for the complete DMA transfer to memory. Each linked list item contains four words:
 - Source address.
 - Destination address.
 - Pointer to next LLI.
 - Control word.

The last LLI has its linked list word pointer set to 0.

2. Choose a free DMA channel with the priority required. DMA channel 0 has the highest priority and DMA channel 7 the lowest priority.
3. Write the first linked list item, previously written to memory, to the relevant channel in the DMA Controller.
4. Write the channel configuration information to the channel Configuration Register and set the Channel Enable bit. The DMA Controller then transfers the first and then subsequent packets of data as each linked list item is loaded.
5. An interrupt can be generated at the end of each LLI depending on the Terminal Count bit in the DMACCxControl Register. If this bit is set an interrupt is generated at the end of the relevant LLI. The interrupt request must then be serviced and the relevant bit in the DMACIntTCClear Register must be set to clear the interrupt.

31.6.5.1.2 Example of scatter/gather DMA

See [Figure 136](#) for an example of an LLI. A section of memory is to be transferred to a peripheral. The addresses of each LLI entry are given, in hexadecimal, at the left-hand side of the figure. In this example, the LLIs describing the transfer are to be stored contiguously from address 0x2002 0000, but they could be located anywhere. The right side of the figure shows the memory containing the data to be transferred.



The first LLI, stored at 0x2002 0000, defines the first block of data to be transferred, which is the data stored from address 0x2002 A200 to 0x2002 ADFF:

- Source start address 0x2002 A200.
- Destination address set to the destination peripheral address.
- Transfer width, word (32-bit).
- Transfer size, 3072 bytes (0xC00).
- Source and destination burst sizes, 16 transfers.
- Next LLI address, 0x2002 0010.

The second LLI, stored at 0x2002 0010, describes the next block of data to be transferred:

- Source start address 0x2002 B200.
- Destination address set to the destination peripheral address.
- Transfer width, word (32-bit).
- Transfer size, 3072 bytes (0xC00).
- Source and destination burst sizes, 16 transfers.
- Next LLI address, 0x2002 0020.

A chain of descriptors is built up, each one pointing to the next in the series. To initialize the DMA stream, the first LLI, 0x2002 0000, is programmed into the DMA Controller. When the first packet of data has been transferred the next LLI is automatically loaded.

The final LLI is stored at 0x2002 0070 and contains:

- Source start address 0x2003 1200.
- Destination address set to the destination peripheral address.
- Transfer width, word (32-bit).
- Transfer size, 3072 bytes (0xC00).
- Source and destination burst sizes, 16 transfers.
- Next LLI address, 0x0.

Because the next LLI address is set to zero, this is the last descriptor, and the DMA channel is disabled after transferring the last item of data. The channel is probably set to generate an interrupt at this point to indicate to the ARM processor that the channel can be reprogrammed.